

WERKSTATTBERICHT · DIGITAL-HUMANITIES-PROJEKT

Werkstattbericht: Zwei verzahnte DaZ-Anwendungen als Digital-Humanities-Projekt

YAMA-Deutsch & YAMA-DaZ-Sprachvergleich — Architektur, Arbeitsablauf, Lernkreislauf

Dr. Ergün Özsoy · LMU München · **Lebendes Dokument** — wird mit dem Projekt fortgeschrieben · Stand: Juli 2026

1. Ziel und Gegenstand

Diese Notiz dokumentiert, wie zwei fachdidaktische Web-Anwendungen ohne Server-Infrastruktur, ohne Budget und ohne Programmierteam entstanden sind — als reproduzierbares Muster für geisteswissenschaftliche Projekte:

- **YAMA-Deutsch** — interaktive Lernanwendung (Einstiegstest mit zweisprachigem Feedback DE/TR, Übungsmuster, Satzprüfer) für Deutsch als Zweitsprache aus türkischer L1-Perspektive.
- **YAMA-DaZ-Sprachvergleich** — wissenschaftliche Plattform zum Sprachvergleich Deutsch–Türkisch: 19 Module in drei Bereichen (Laut & Schrift, Morphologie, Syntax), jedes im festen Fünf-Schritte-Raster (Kontrastiver Befund → Didaktik → Interferenzanalyse → Korrektur & Förderung → Mehrsprachigkeit als Ressource), dazu unterrichtsfertige Arbeitsblätter und eine kommentierte Bibliografie.

Leitidee: **Fachwissen zuerst, Technik als Dienerin**. Die kontrastive Linguistik bestimmt die Struktur; die Technik bleibt so einfach wie möglich.

2. Architektur im Überblick

2.1 Zwei bewusst verschiedene Bauweisen

	DaZ-Sprachvergleich	YAMA-Deutsch
Typ	Eine einzige HTML-Datei	React/TypeScript-App (Vite)
Inhalte	eingebettetes JSON-Objekt (const APP)	drei externe JSON-Dateien
Build	keiner — Datei ist die Anwendung	GitHub Actions baut automatisch
Hosting	GitHub Pages	GitHub Pages (via Actions)
Pflegeaufwand	minimal	mittel

Warum zwei Bauweisen? Die Sprachvergleich-Plattform ist primär ein *Lese- und Nachschlagewerk* — dafür genügt eine Datei, die überall läuft und archivierbar bleibt (Nachhaltigkeit!). Die Lernanwendung dagegen braucht Interaktionslogik (Testauswertung, Aggregation, Empfehlungen) — dafür lohnt ein Framework.

2.2 Das wichtigste Prinzip: Trennung von Inhalt und Code

In YAMA-Deutsch liegen alle fachlichen Inhalte in drei JSON-Dateien:

- `errors.json` — 24 dokumentierte Interferenzfehler (mit Ursache, Typ interlingual/intralingual, zweisprachigem Feedback)
- `diagnostic_tasks.json` — die Testaufgaben des Einstiegstests
- `exercise_templates.json` — die Übungsmuster (Anweisung, Beispiel, erwartete Antwort, Korrekturmuster)

Folge: Neue Übungen erfordern keine Programmierung. Als eine kollegiale Rückmeldung drei neue textbezogene Aufgaben vorschlug, wurden sie als reine Dateneinträge ergänzt — der Code blieb unberührt, die Oberfläche zeigte sie automatisch an. Dies ist das DH-Prinzip *data-driven design*: Die Fachperson pflegt Daten, nicht Code.

3. Werkzeuge und Infrastruktur (alles kostenfrei)

1. **GitHub** — Versionierung und „Single Source of Truth“: Jede Änderung ist dokumentiert, datiert, rückholbar.
2. **GitHub Pages** — Hosting statischer Seiten direkt aus dem Repository; keine Serverkosten, keine Wartung.
3. **GitHub Actions** — automatischer Build: Nach jedem Commit wird YAMA-Deutsch neu kompiliert und veröffentlicht (ca. 1–2 Minuten).
4. **Vite + React + TypeScript** — Framework der Lernanwendung; TypeScript fängt Fehler vor der Veröffentlichung ab.
5. **Python + WeasyPrint** — Generator für die 24 PDF-Dossiers und Arbeitsblätter: Inhalte liegen in einer Python-Datei (`content.py`), das Skript (`gen_pdfs.py`) erzeugt daraus einheitlich gestaltete PDFs. Eine Korrektur an einer Stelle wirkt in alle Dokumente — *ein* Quelltext, viele Ausgaben.
6. **KI-Assistenz (Claude, Anthropic)** — als Implementierungswerkzeug unter fachlicher Steuerung (siehe Abschnitt 6).

4. Arbeitsablauf Schritt für Schritt

Der wiederkehrende Zyklus, mit dem sämtliche Erweiterungen umgesetzt wurden:

1. **Fachliche Entscheidung** (Mensch): Was soll entstehen — welches Modul, welche Übung, welche Verknüpfung?
2. **Ist-Stand einlesen**: Die aktuelle Datei wird direkt aus dem Repository geholt — *nie* aus dem Gedächtnis oder einer lokalen Kopie überschrieben. (Regel: „Das Repo ist die Wahrheit.“)
3. **Änderung minimal-invasiv einarbeiten**: Nur ergänzen, Bestehendes nicht umbauen; jede Änderung wird vorab exakt lokalisiert.
4. **Validieren vor dem Hochladen**: JSON-Parsing, JavaScript-Syntaxprüfung, bei TypeScript ein Compiler-Check — Fehler werden abgefangen, bevor sie live gehen.
5. **Veröffentlichen per „Edit + Paste“**: Auf GitHub die Zielfeile öffnen → Bearbeiten → Inhalt ersetzen → Commit. (Datei-Upload per Drag & Drop vermeiden — er erzeugt leicht Duplikate wie `index (1).html`.)

6. **Verifizieren:** Nach dem Commit wird die Datei erneut aus dem Repository gelesen und mit der Sollversion byte-genau verglichen; bei YAMA-Deutsch zusätzlich: Actions-Lauf grün? Live-Seite aktualisiert?

Dieser Zyklus — *lesen* → *ändern* → *prüfen* → *veröffentlichen* → *verifizieren* — ist bewusst konservativ. Er hat sich als verlässlich erwiesen, gerade weil keine lokale Entwicklungsumgebung vorausgesetzt wird: Alles geschieht über den Browser.

5. Fallstudie: Der geschlossene Lernkreislauf

Eine kollegiale Rückmeldung (Juli 2026) lieferte zwei Impulse: (a) Übungen von der Einzelsatz- auf die **Textebene** heben; (b) beide Anwendungen **querverweisen** — „Diagnose, Erklärung und Übung als geschlossener Lernkreislauf“.

Umsetzung am selben Tag, in drei Schritten:

1. **Vier neue Übungsmuster** (V2 im Tagesablauf, Vorgangspassiv im Rezept, Formeln im Dialog, Kommasetzung im Einladungstext) — als reine Daten-Ergänzung in `exercise_templates.json`, im vorhandenen Format, in den vorhandenen Kategorien (SYN, TEXT, PRAG).
2. **Deep-Links in der Sprachvergleich-Plattform:** Acht Zeilen JavaScript genügen, damit jedes Modul eine eigene Adresse erhält (`.../#wortstell`, `.../#genus`, `.../#passiv`). Module werden damit *zitierfähig* und *verlinkbar* — auch für Lehre und E-Mail.
3. **Verknüpfung in der Lernanwendung:** Eine Zuordnungstabelle (Interferenzfehler → passendes Modul) und ein bedingter Link auf den Empfehlungskarten des Einstiegstests: „Zum Sprachvergleich-Modul →“. Wer z. B. an der Verbklammer scheitert, landet mit einem Klick im Modul „Wortstellung (SOV vs. V2)“.

Ergebnis: **Diagnose (Test) → Erklärung (Modul) → Übung (Textaufgabe)** — der Kreislauf ist geschlossen. Bemerkenswert aus DH-Sicht: Schritt 1 war reine Datenpflege, Schritt 2 acht Zeilen, Schritt 3 eine kleine, typgeprüfte Code-Ergänzung. Gute Architektur macht gute Ideen billig.

6. KI-Einsatz und Transparenz

Die technische Umsetzung erfolgte in Zusammenarbeit mit einem KI-Assistenten (Claude, Anthropic): Einlesen der Ist-Stände, Einarbeiten der Änderungen, Syntax- und Typprüfungen, Verifikation nach der Veröffentlichung. Die **fachlichen Inhalte, didaktischen Entscheidungen und die Qualitätskontrolle** liegen durchgehend beim Autor; die KI fungierte als Werkzeug der Implementierung — vergleichbar einem sehr schnellen, aber stets zu beaufsichtigenden Assistenten. Dieses Vorgehen folgt den einschlägigen Transparenzempfehlungen (COPE; DFG; Europäische Kommission) zum KI-Einsatz in Forschung und Lehre.

7. Lessons Learned — sieben Grundsätze

1. **Inhalt von Code trennen** — Fachpflege ohne Programmierung ermöglichen.
2. **Das Repository ist die einzige Wahrheit** — vor jeder Änderung den Ist-Stand einlesen.
3. **Nur ergänzen, nicht umbauen** — minimal-invasive Änderungen halten das Risiko klein.
4. **Vor dem Hochladen validieren** — Syntaxfehler dürfen die Live-Seite nie erreichen.
5. **Nach dem Hochladen verifizieren** — Vertrauen ist gut, byte-genauer Vergleich ist besser.
6. **Adressierbarkeit schaffen** — Deep-Links machen wissenschaftliche Inhalte zitier- und verlinkbar.

7. **Kollegiales Feedback als Motor** — die wertvollsten Erweiterungen kamen von außen; schnelle Umsetzung honoriert gute Rückmeldung.

8. Ausblick

Geplant sind: Einbindung beider Anwendungen als ergänzende Materialien in die persönliche LMU-Seite (userweb.mwn.de); Erweiterung der Textebene (kohärente Kurztexte als Standard-Übungsformat); Ausbau der Bibliografie; perspektivisch Analytik zur anonymen Nutzungsauswertung.

9. Änderungsjournal

Datum	Änderung
Juli 2026	Erstfassung des Werkstattberichts. Vier textbezogene Übungsmuster (SYN/TEXT/PRAG) ergänzt; Deep-Links (<code>#modulId</code>) in der Sprachvergleich-Plattform; Lernkreislauf-Verknüpfung im Einstiegstest („Zum Sprachvergleich-Modul →“).
Juni 2026	PDF-Generator (WeasyPrint) vereinheitlicht; 24 Dossiers/Arbeitsblätter neu erzeugt.

Neue Einträge oben anfügen — eine Zeile pro Entwicklungsschritt genügt.

Beide Anwendungen: ergunozsoy.github.io/YAMA-Deutsch · ergunozsoy.github.io/YAMA-DaZ-Sprachvergleich — Quellcode offen einsehbar auf github.com/ergunozsoy. Dieses Dokument liegt in beiden Repositories als `YAMA_WERKSTATT.md`.